

cTheorems

sahasatvik

<https://github.com/sahasatvik/typst-theorems>

Contents

1. Introduction	1
2. Usage	1
3. Features	2
3.1. Proofs	2
3.1.1. Equation tags	3
4. Acknowledgements	3
5. Function documentation	4

1. Introduction

This package provides functions that help create numbered theorem environments. This is heavily inspired by the L^AT_EX packages `amsthm` and `thmtools`.

A *theorem environment* lets you wrap content together with automatically updating *numbering* information. Such environments use internal `state` counters for this purpose. Environments can

- share the same counter (*Theorems* and *Lemmas* often do so)
- keep a global count, or be attached to
 - other environments (*Corollaries* are often numbered based upon the parent *Theorem*)
 - headings
- have a numbering level depth fixed (for instance, use only top level heading numbers)
- be referenced elsewhere in the document, via `@references` and `<label>s`
- be restated or deferred to later in the document, via `thm-restate`

This package also adds a couple of miscellaneous features:

- *QED* symbols are inserted at the end of proofs, and can be positioned via `qedhere`.
- Equations can be tagged (as in L^AT_EX's `\tag`).

2. Usage

Import all functions provided by `typst-theorems` using

```
#import "@preview/ctheorems:2.0.0": *
#show: thm-rules // Must include!
```

Including the show rule in the second line is crucial for displaying theorem environments, references, and equations correctly! All of the examples in this document will presume that `thm-rules` has been applied.

```
#import thm-themes.standard: *
```

The core of this module consists of `thm-env`. Functions like `thm-plain`, `thm-def`, `thm-rem`, and `thm-proof` provide some simple defaults for the appearance of `thm-envs`, analogous to the styles in `amsthm`. All theorem environment data is stored in the state `thm-stored`. This is used by functions

like `thm-restate` and `thm-display` to manipulate and display theorem environments elsewhere in the document.

3. Features

We will create a theorem using `thm-plain`, and use it as follows.

```
#let theorem = thm-plain("Theorem")

#theorem("Euclid")[
  There are infinitely many primes.
] <euclid>
```

Theorem 3.1 (Euclid). *There are infinitely many primes.*

Note that this theorem inherits its numbering from the heading by default. Since this theorem has been labeled `<euclid>`, we can reference it elsewhere in the document via `@euclid`.

We will supply a proof of `@euclid` later.

We will supply a proof of Theorem 3.1 later.

3.1. Proofs

Theorems go hand in hand with proofs.

```
#let proof = thm-proof("Proof")

#proof([of @euclid])[
  Suppose to the contrary that $p_1, p_2,
  dots, p_n$ is a finite enumeration of
  all primes. Set $P = p_1 p_2 \dots p_n$.
  Since $P + 1$ is not in our list, it cannot
  be prime. Thus, some prime factor $p_j$
  divides $P + 1$. Since $p_j$ also divides
  $P$, it must divide the difference $(P + 1)
  - P = 1$, a contradiction.
]
```

Proof of Theorem 3.1. Suppose to the contrary that $p_1, p_2, \dots, p_n$ is a finite enumeration of all primes. Set $P = p_1 p_2 \dots p_n$. Since $P + 1$ is not in our list, it cannot be prime. Thus, some prime factor p_j divides $P + 1$. Since p_j also divides P , it must divide the difference $(P + 1) - P = 1$, a contradiction. ■

Proof environments will float a *QED* symbol (■) at the bottom right by default. The symbol can be customized by setting `thm-rules.qed-symbol`.

```
#show: thm-rules.with(qed-symbol: $square$)
```

If your proof ends in a block equation, or a list/enum, you can place `qedhere` to correctly position the qed symbol.

```
#theorem[
  There are arbitrarily long stretches
  of composite numbers.
]
#proof[
  For any $n > 2$, consider $
    n! + 2, quad
    n! + 3, quad ..., quad
    n! + n #qedhere
$
]
```

Theorem 3.1.1. *There are arbitrarily long stretches of composite numbers.*

Proof. For any $n > 2$, consider

$$n! + 2, \quad n! + 3, \quad \dots, \quad n! + n \quad \blacksquare$$

Caution: [qedhere](#) does not play well with numbered equations!

3.1.1. Equation tags

You can also insert tags into equations that float to the right, mimicking `\tag` in $\text{\LaTeX}$.

```
$
(a + b)^2
&= a^2 + 2 a b + b^2 \
&\leq 2a^2 + 2b^2 #tag[(AM-GM)] \
&\leq 2 \thickmax{a, b}^2.
$
```

$$\begin{aligned}(a + b)^2 &= a^2 + 2ab + b^2 \\ &\leq 2a^2 + 2b^2 && \text{(AM-GM)} \\ &\leq 2 \max\{a, b\}^2.\end{aligned}$$

This is essentially how the *QED* symbol is placed inside equations by [qedhere](#).

```
#let theorem-standout = theorem.with(
  stroke: 1pt,
  outset: 0.7em,
  padding: (y: 1em)
)

#theorem-standout("Important")[#lorem(6)]
#lorem(8)
#remark[#lorem(4)]
#corollary[#lorem(2)]
#corollary[#lorem(4)]
```

Theorem 3.1.1.1 (Important). *Lorem ipsum dolor sit amet, consectetur.*

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Remark. Lorem ipsum dolor sit.

Corollary 3.1.1.1.1. *Lorem ipsum.*

Corollary 3.1.1.1.2. *Lorem ipsum dolor sit.*

4. Acknowledgements

Thanks to

- [MJHutchinson](#) for suggesting and implementing the `base-level` and `base: none` features,
- [rmolinari](#) for suggesting and implementing the `separator: ...` feature,
- [DVDTSB](#) for contributing
 - the idea of passing named arguments from the theorem directly to the `fmt` function.
 - the `number: ...` override feature.
 - the `title: ...` override feature in `thm-plain`.
- [PgBiel](#) for fixing breaking changes in version updates.
- The awesome devs of [typst.app](#) for their support.

5. Function documentation

- [thm-rules\(\)](#)
- [thm-env\(\)](#)
- [thm-box\(\)](#)
- [thm-plain\(\)](#)
- [thm-def\(\)](#)
- [thm-rem\(\)](#)
- [thm-proof\(\)](#)
- [proof-body-fmt\(\)](#)
- [tag\(\)](#)
- [thm-restate\(\)](#)
- [thm-display\(\)](#)

Variables:

- [thm-counters](#)
- [thm-stored](#)
- [qedhere](#)

thm-rules

Rules for styling theorem environments, references, proofs, etc. *Must appear at the beginning of the document.*

Parameters

```
thm-rules(  
  qed-symbol: content,  
  doc: content  
) -> content
```

qed-symbol `content`

Symbol displayed at the end of proofs. See [thm-proof\(\)](#), [qedhere](#), [proof-body-fmt\(\)](#). Use as

```
#show: thm-rules.with(  
  qed-symbol: $square$  
)  
  
#let proof = thm-proof("Proof")  
  
#proof[#lorem(3)]  
#proof[  
  #lorem(5)  
  $ integral_0^oo sin(x)/x = pi/2. #qedhere $  
]
```

Default: `$qed$`

Proof. Lorem ipsum dolor. □

Proof. Lorem ipsum dolor sit amet.

$$\int_0^\infty \frac{\sin(x)}{x} = \frac{\pi}{2}. \quad \square$$

doc `content`

Document

thm-env

Creates a theorem environment, which is a function of the form

```
(
  ..thm-args,
  body,
  number: auto,
  numbering: "1.1",
  base: base,
  base-level: base-level,
  restate: false,
  defer: false,
  restate-keys: (counter, ),
  supplement: counter,
  ref-fmt: (supplement, thm) => {
    if supplement ≠ none { supplement = [#supplement~] }
    [#supplement#link(thm.loc, (thm.number))]}
  },
) → content
```

The `body` contains the content of the theorem environment, and `thm-args` get passed to the formatting function `fmt`. The first positional argument from `thm-args` is interpreted as the name of the theorem environment.

The `numbering` option specifies the numbering used for the theorem environment (set to `none` for turning numbering off). Setting the `number` option lets you override the automatic numbering with content.

The `base` and `base-level` options are inherited from the `thm-env` call; see the list of parameters below.

The `supplement` determines the default supplement used when a labeled theorem environment is referenced. The `ref-fmt` lets you specify custom formatting for references; see [thm-display\(\)](#) for more details on the `thm` dictionary.

See [thm-restate\(\)](#) for more information about the `restate`, `defer`, and `restate-keys` options.

```
#let theorem = thm-env(
  "Theorem",
  fmt: (name, number, body, color: black, ..args)
=> {
  if name ≠ none { name = [~(#name)] }
  text(color)[
    *Theorem~#number*~#name:~#body\
  ]
},
base: "heading"
)
```

```
= First_heading
#theorem[#lorem(5)]
#theorem("Named")[#lorem(7)]
```

Refer to @thm.

```
= First_Subheading
#theorem[#lorem(3)]
#theorem[#lorem(4)]

= Second_Subheading
#theorem[#lorem(6)]
#theorem(color: red)[#lorem(2)] <thm>
```

```
= Second_heading
#theorem[#lorem(4)]
#theorem(number: $dagger$)[#lorem(9)]
#theorem[#lorem(7)]
```

1 First heading

Theorem 1.1: Lorem ipsum dolor sit amet.

Theorem 1.2 (Named): Lorem ipsum dolor sit amet, consectetur adipiscing.

Refer to Theorem 1.2.2.

1.1 First Subheading

Theorem 1.1.1: Lorem ipsum dolor.

Theorem 1.1.2: Lorem ipsum dolor sit.

1.2 Second Subheading

Theorem 1.2.1: Lorem ipsum dolor sit amet, consectetur.

Theorem 1.2.2: Lorem ipsum.

2 Second heading

Theorem 2.1: Lorem ipsum dolor sit.

Theorem †: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed.

Theorem 2.2: Lorem ipsum dolor sit amet, consectetur adipiscing.

Parameters

```
thm-env(
  counter: string,
  fmt: function,
  base: string none,
  base-level: int none
) -> function
```

counter string

Environment counter name

fmt function

Formatting function, of the form (name, number, body, ..fmt-args) → content. When a theorem environment is called, the named arguments from thm-args are passed into fmt-args.

Default: thm-fmt-default

base string or none

Base counter name, whose numbering prefixes the theorem environment numbering. If none, the theorem environment maintains a global count with no prefix.

Default: **none**

base-level **int** or **none**

Base level, determining the number of levels of the **base** numbering to use during the theorem environment numbering. If **none**, all levels from the **base** numbering are used.

Default: **none**

thm-box

Creates a theorem environment wrapped in a padded block, with sensible default styling. The block has width: 100% applied by default. The `fmt` function is of the form `(name, number, body, title: auto, ..fmt-args) -> content`. All named arguments from `args`, followed by all named `fmt-args`, are passed to the `block` call.

```
#let notation = thm-box(  
  "Notation",  
  base: none,  
  numbering: "I",  
  title-fmt: t => smallcaps(strong(t)),  
  body-fmt: emph,  
  outset: 0.7em,  
  padding: (y: 0.5em),  
  radius: 2pt,  
  fill: rgb("#d4e2fe"),  
)  
  
#lorem(5)  
#notation[#lorem(3)]  
#notation[#lorem(7)]
```

Lorem ipsum dolor sit amet.

NOTATION I. *Lorem ipsum dolor.*

NOTATION II. *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Parameters

```
thm-box(  
  head: content,  
  counter: string auto,  
  ..args: any,  
  numbering: string function,  
  supplement: string auto,  
  padding: dictionary,  
  name-fmt: function,  
  title-fmt: function,  
  body-fmt: function,  
  separator: content,  
  base: string,  
  base-level: int none  
) -> function
```

head **content**

Environment heading.

counter **string** or **auto**

Environment counter name. If `auto`, set to `head`.

Default: `auto`

`..args` `any`

Named arguments for the block.

`numbering` `string` or `function`

Environment numbering style.

Default: `"1.1"`

`supplement` `string` or `auto`

Supplement for references. If `auto`, set to `head`.

Default: `auto`

`padding` `dictionary`

Padding around the block.

Default: `(y: 0.1em)`

`name-fmt` `function`

Formatting for the environment name.

Default: `x ⇒ [(#x)]`

`title-fmt` `function`

Formatting for the environment title (head and number).

Default: `x ⇒ x`

`body-fmt` `function`

Formatting for the environment body.

Default: `x ⇒ x`

`separator` `content`

Separator between title and body.

Default: `[. ]`

`base` `string`

Base counter name.

Default: "heading"

base-level `int` or `none`

Base level.

Default: `none`

thm-plain

Creates a plain theorem environment, suitable for theorems, lemmas, corollaries, propositions and conjectures. Identical to `thm-box()`, with different defaults.

```
#let theorem = thm-plain(  
  "Theorem",  
  base: none  
)  
  
#let lemma = thm-plain(  
  "Lemma",  
  counter: "Theorem",  
  base: none  
)  
  
#let corollary = thm-plain(  
  "Corollary",  
  base: "Theorem"  
)  
  
#lemma[#lorem(3)]  
#theorem("Named")[#lorem(4)]  
#corollary[#lorem(7)]  
#theorem[#lorem(7)]
```

Lemma 1. *Lorem ipsum dolor.*

Theorem 2 (Named). *Lorem ipsum dolor sit.*

Corollary 2.1. *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Theorem 3. *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Parameters

`thm-plain()` -> `function`

thm-def

Creates a theorem environment, suitable for definitions, conditions, problems and examples. Identical to `thm-box()`, with different defaults.

```
#let definition = thm-def(  
  "Definition",  
  base: none  
)  
  
#definition[#lorem(7)]  
#definition[#lorem(4)]
```

Definition 1. Lorem ipsum dolor sit amet, consectetur adipiscing.

Definition 2. Lorem ipsum dolor sit.

Parameters

`thm-def()` -> `function`

thm-rem

Creates a theorem environment, suitable for remarks, notes, annotations, claims, cases, acknowledgments and conclusions. Identical to `thm-box()`, with different defaults.

```
#let remark = thm-rem(  
  "Remark",  
  base: none  
)  
  
#remark[#lorem(3)]  
#remark[#lorem(6)]
```

Remark. Lorem ipsum dolor.

Remark. Lorem ipsum dolor sit amet, consectetur.

Parameters

`thm-rem()` -> `function`

thm-proof

Creates a proof environment Identical to `thm-rem()`, with different defaults.

```
#let theorem = thm-plain(  
  "Theorem",  
  base: none  
)  
#let proof = thm-proof("Proof")  
  
#theorem[#lorem(6)]  
#proof[#lorem(3)]
```

Theorem 1. Lorem ipsum dolor sit amet, consectetur.

Proof. Lorem ipsum dolor. ■

Parameters

`thm-proof()` -> `function`

proof-body-fmt

Used as the `body-fmt` in `thm-proof()`, for properly styling proofs by inserting a `qed` symbol at the end of the body. Also see [qedhere](#).

```
#show: thm-rules.with(qed-symbol: $"Q.E.D."$)  
  
#proof-body-fmt[#lorem(3)]  
#v(2em)  
  
#proof-body-fmt[  
  $  
  phi.alt(x) = 1/sqrt(2 pi) e^(-x^2/2) #qedhere  
  $  
]
```

Lorem ipsum dolor. Q.E.D.

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \quad \text{Q.E.D.}$$

Parameters

`proof-body-fmt(body: content)` -> `content`

body `content`

Proof body.

tag

Add content which floats to the right in an equation.

```
$
(a + b)^2
&= a^2 + 2 a b + b^2 \
&\leq 2a^2 + 2b^2 #tag[(AM-GM)] \
&\leq 2 \thin \max{a, b}^2.
$
```

$$\begin{aligned}(a + b)^2 &= a^2 + 2ab + b^2 \\ &\leq 2a^2 + 2b^2 & (\text{AM-GM}) \\ &\leq 2 \max\{a, b\}^2.\end{aligned}$$

Parameters

`tag(t: any) -> content`

t any

Content to use as tag

thm-restate

Displays theorem environments which have been marked to be restated or deferred, can be filtered. Useful for pushing content to the appendix. See `thm-display()` for the structure of a `thm`.

The following example illustrates the basic usage of `thm-restate`, combined with the `restate` and `defer` flags for theorem environments.

```
#let theorem = thm-plain("Theorem")
#let lemma = thm-plain(
  "Lemma",
  counter: "Theorem",
)
#let definition = thm-def("Definition")
#let proof = thm-proof("Proof")

≡ Heading

#definition[#lorem(2)]
#lemma[#lorem(8)]

#theorem("Name", restate: true)[#lorem(7)]
#proof(defer: true)[
  #lorem(7)
]

#lemma[#lorem(4)]

≡ Appendix

#thm-restate()
```

1 Heading

Definition 1.1. Lorem ipsum.

Lemma 1.1. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Theorem 1.2 (Name). Lorem ipsum dolor sit amet, consectetur adipiscing.

Lemma 1.3. Lorem ipsum dolor sit.

2 Appendix

Theorem 1.2 (Name). Lorem ipsum dolor sit amet, consectetur adipiscing.

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Parameters

```
thm-restate(
  ..keys: string array function,
  fmt: function auto,
  at: label selector location function auto,
  final: boolean
) -> content
```

..keys `string` or `array` or `function`

String keys, array of keys, or functions used to filter theorem environments. A thm is displayed if it passes *any* of the filters.

If `k` in `keys` is a `string`, theorem environments containing `k` in its array of `restate-keys` will be matched.

```
#let theorem = thm-plain("Theorem")
#let lemma = thm-plain(
  "Lemma",
  counter: "Theorem",
)
#let proof = thm-proof("Proof")

= Heading

#theorem(restate: true)[#lorem(6)]
#lemma(restate: true)[#lorem(4)]
#proof(defer: true)[#lorem(7)]
#lemma(restate: true)[#lorem(3)]

= Restate lemmas/proofs
#thm-restate("Lemma", "Proof")
```

1 Heading

Theorem 1.1. *Lorem ipsum dolor sit amet, consectetur.*

Lemma 1.2. *Lorem ipsum dolor sit.*

Lemma 1.3. *Lorem ipsum dolor.*

2 Restate lemmas/proofs

Lemma 1.2. *Lorem ipsum dolor sit.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Lemma 1.3. *Lorem ipsum dolor.*

If `k` in `keys` is an array of `strings`, theorem environments containing *all* keys from `k` in its array of `restate-keys` will be matched.

```
= Heading

#theorem(
  "Result A",
  restate: true,
  restate-keys: ("Theorem", "Result A")
)[#lorem(6)]
#proof(
  defer: true,
  restate-keys: ("Proof", "Result A")
)[#lorem(7)]
#theorem(restate: true)[#lorem(6)]
#theorem(
  "Result B",
  restate: true,
  restate-keys: ("Theorem", "Result B")
)[#lorem(6)]
#proof(
  defer: true,
  restate-keys: ("Proof", "Result B")
)[#lorem(7)]

= Restate Result A
#thm-restate("Result A")

= Restate theorems tagged Result B
#thm-restate(("Theorem", "Result B"))
```

3 Heading

Theorem 3.1 (Result A). *Lorem ipsum dolor sit amet, consectetur.*

Theorem 3.2. *Lorem ipsum dolor sit amet, consectetur.*

Theorem 3.3 (Result B). *Lorem ipsum dolor sit amet, consectetur.*

4 Restate Result A

Theorem 3.1 (Result A). *Lorem ipsum dolor sit amet, consectetur.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

5 Restate theorems tagged Result B

Theorem 3.3 (Result B). *Lorem ipsum dolor sit amet, consectetur.*

If `k` in `keys` is a function, it must be of the form `restate-keys → boolean`.

```
= Heading

#theorem(
  restate: true,
  restate-keys: (
    "Theorem", "Unproven claim"
  )
)[#lorem(6)]
#theorem(restate: true)[#lorem(6)]
#lemma(
  "Claim D",
  restate: true,
  restate-keys: ("Lemma", "Claim D")
)[#lorem(6)]

= Restate claims
#thm-restate(
  keys ⇒ keys.any(
    k ⇒ lower(k).contains("claim")
  )
)
```

6 Heading

Theorem 6.1. *Lorem ipsum dolor sit amet, consectetur.*

Theorem 6.2. *Lorem ipsum dolor sit amet, consectetur.*

Lemma 6.3 (Claim D). *Lorem ipsum dolor sit amet, consectetur.*

7 Restate claims

Theorem 6.1. *Lorem ipsum dolor sit amet, consectetur.*

Lemma 6.3 (Claim D). *Lorem ipsum dolor sit amet, consectetur.*

fmt `function` or `auto`

Formatting function of the form `thm → content`. The default `auto` uses the same `fmt` originally supplied to the `thm-env`. See `thm-display.fmt`.

Default: `auto`

at `label` or `selector` or `location` or `function` or `auto`

Location up to which theorem environments will be displayed. The default `auto` uses the location where `thm-restate` was called. See `thm-display.at`.

Default: `auto`

final `boolean`

If `true`, display environments up to the end of the document. See `thm-display.final`.

Default: `false`

thm-display

Displays all theorem environments, can be filtered. A `thm` is a dictionary storing information about a theorem environment, with keys

```
(
  args,
  name,
  body,
```

```

supplement,
fmt,
number,
numbering,
restate,
defer,
restate-keys,
ref-fmt,
loc,
counter,
base,
base-level
)

```

These contain information supplied to the theorem environment when created/called, and can be used to reconstruct it completely.

```

#let theorem = thm-plain("Theorem")
#let lemma = thm-plain(
  "Lemma",
  counter: "Theorem",
)
#let definition = thm-def("Definition")
#let proof = thm-proof("Proof")

= Heading <h1>

#theorem("Name")[#lorem(7)]
#proof[
  #lorem(7)
]

= New heading <h2>

#lemma[#lorem(8)]
#definition("Thing")[#lorem(2)]
#lemma[#lorem(4)]

= Display all

#thm-display()

```

1 Heading

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

2 New heading

Lemma 2.1. *Lorem ipsum dolor sit amet, consectetur adipiscing elit.*

Definition 2.1 (Thing). Lorem ipsum.

Lemma 2.2. *Lorem ipsum dolor sit.*

3 Display all

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Lemma 2.1. *Lorem ipsum dolor sit amet, consectetur adipiscing elit.*

Definition 2.1 (Thing). Lorem ipsum.

Lemma 2.2. *Lorem ipsum dolor sit.*

The key `loc` gives the location of the theorem environment in the document. The `number` gives the (calculated and formatted) number of the theorem environment. The remaining keys contain information as detailed in `thm-env()`.

Parameters

```

thm-display(
  ..filters: function,
  fmt: function auto,
  at: label selector location function auto,

```

```

final: boolean
) -> content

```

..filters function

Filtering functions. Each `f` in `filters` is a function `thm → boolean`. A `thm` is displayed if it passes *any* of the filters.

```

= Display only theorems/proofs

#thm-display(
  thm => thm.supplement == "Theorem",
  thm => thm.supplement == "Proof",
)

= Display if 'name' is present

#thm-display(
  thm => thm.name != none
)

```

4 Display only theorems/proofs

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

5 Display if name is present

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Definition 2.1 (Thing). Lorem ipsum.

fmt function or auto

Formatting function of the form `thm → content`. The default `auto` uses the same `fmt` originally supplied to the `thm-env`.

```

= List of things

#thm-display(
  thm => thm.supplement != "Proof",
  final: true,
  fmt: thm => {
    let head = [*thm.supplement~#thm.number*]
    if thm.name != none {
      head = head + [~(#thm.name)]
    }
    let page = thm.loc.position().page
    let page = link(thm.loc, [#page])
    [#head~#box(width: 1fr, repeat[.])~#page\ ]
  }
)

```

6 List of things

Theorem 1.1 (Name)	14
Lemma 2.1	14
Definition 2.1 (Thing)	14
Lemma 2.2	14

The `final: true` ensures that even if this `thm-display` call is placed at the beginning of the document, all theorem environments are listed.

Default: `auto`

at label or selector or location or function or auto

Location up to which theorem environments will be displayed. The default `auto` uses the location where `thm-display` was called.

```
= Display up to '<h2>'  
#thm-display(at: <h2>)
```

7 Display up to <h2>

Theorem 1.1 (Name). *Lorem ipsum dolor sit amet, consectetur adipiscing.*

Proof. Lorem ipsum dolor sit amet, consectetur adipiscing. ■

Default: **auto**

final **boolean**

If **true**, display all theorem environments up to the end of the document. Useful for creating lists of theorems in the beginning of documents, before they've been stated. Overrides **at**.

Default: **false**

thm-counters **state**

Theorem environment counters

thm-stored **state**

State containing theorem environment data, as an array of **thm** dictionaries. See [thm-display\(\)](#) for details on the structure of each **thm**.

qedhere **metadata**

If placed in a block equation/enum/list within a proof, place a qed symbol to its right.


```

#let proof = thm-proof("Proof")

#proof[
  #lorem(3)
  $ x^2 + y^2 = z^2. #qedhere $
]

#proof[
  + #lorem(4)
  + #lorem(5) #qedhere
]

#proof[
  $
    (a + b)^2 &= (a + b)(a + b) \
    &= a^2 + 2 a b + b^2. #qedhere
  $
]

```

Proof. Lorem ipsum dolor.

$$x^2 + y^2 = z^2. \quad \blacksquare$$

Proof.

1. Lorem ipsum dolor sit.
2. Lorem ipsum dolor sit amet. ■

Proof.

$$\begin{aligned}
 (a + b)^2 &= (a + b)(a + b) \\
 &= a^2 + 2ab + b^2. \quad \blacksquare
 \end{aligned}$$